

Market Segmentation

Problem

We have given Ratings submitted by Customers and the Rating of their Loyalty based on their Purchase, now we have to separate them in different groups to make marketing strategy for them.

Importing Libraries required

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import seaborn as sns
sns.set()
from sklearn.cluster import KMeans
```

Loading data

```
data=pd.read_csv(r'C:\\Users\\DELL\\Desktop\\Market Segmentation.csv')
```

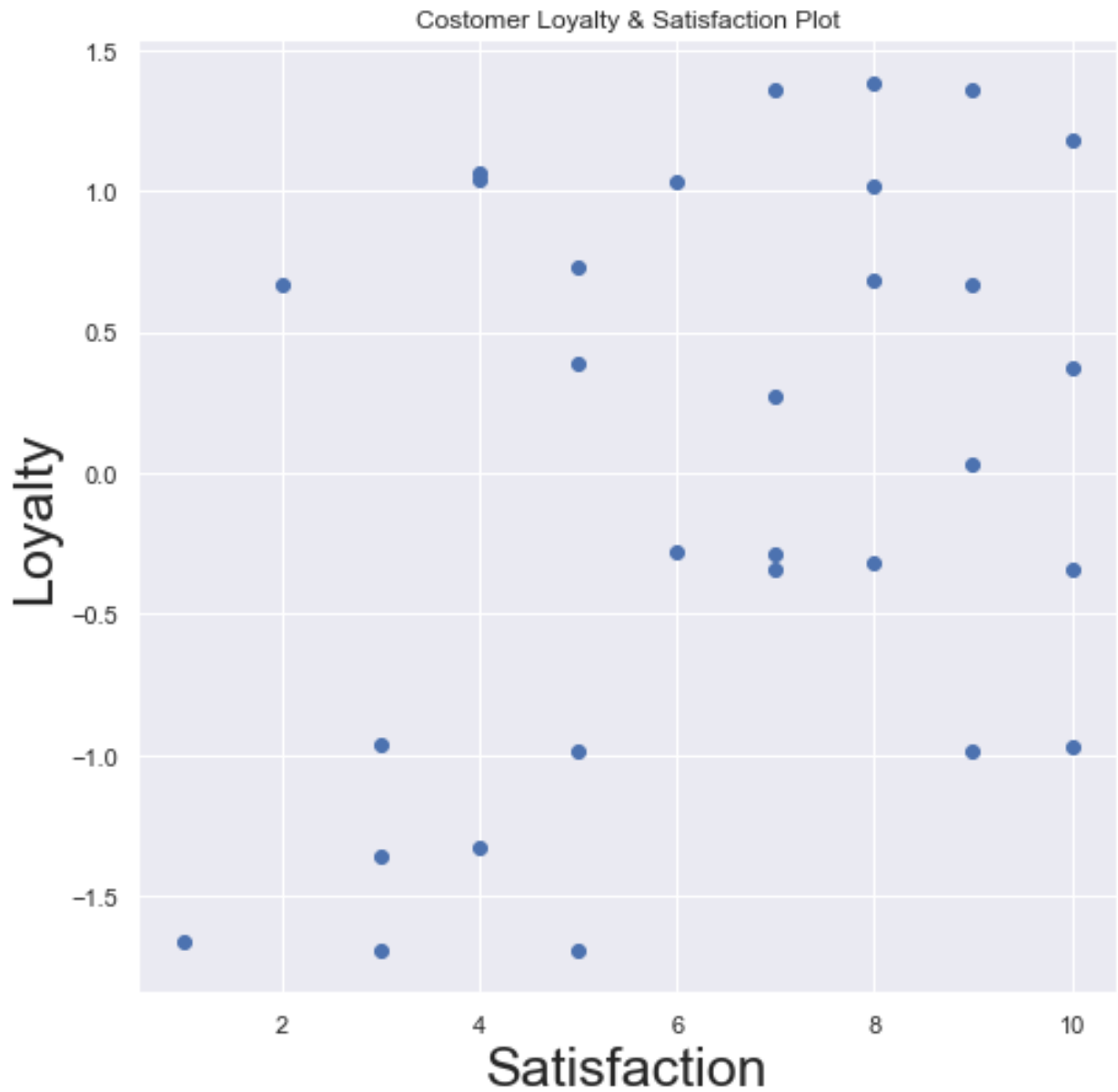
data

	Satisfaction	Loyalty
0	4	-1.33
1	6	-0.28
2	5	-0.99
3	7	-0.29
4	4	1.06
5	1	-1.66
6	10	-0.97
7	8	-0.32
8	8	1.02
9	8	0.68
10	10	-0.34
11	5	0.39
12	5	-1.69
13	2	0.67
14	7	0.27
15	9	1.36
16	8	1.38
17	7	1.36
18	7	-0.34
19	9	0.67
20	10	1.18
21	3	-1.69

22	4	1.04
23	3	-0.96
24	6	1.03
25	9	-0.99
26	10	0.37
27	9	0.03
28	3	-1.36
29	5	0.73

Plot the Data

```
plt.figure(figsize=(8,8))
plt.scatter(data['Satisfaction'],data['Loyalty'])
plt.xlabel('Satisfaction',fontsize = 25)
plt.ylabel('Loyalty',fontsize = 25)
plt.title('Costomer Loyalty & Satisfaction Plot')
Text(0.5, 1.0, 'Costomer Loyalty & Satisfaction Plot')
```



Select the features

```
x=data.copy()
```

Clustering

```
kmeans=KMeans()  
kmeans.fit(x)
```

```
KMeans()
```

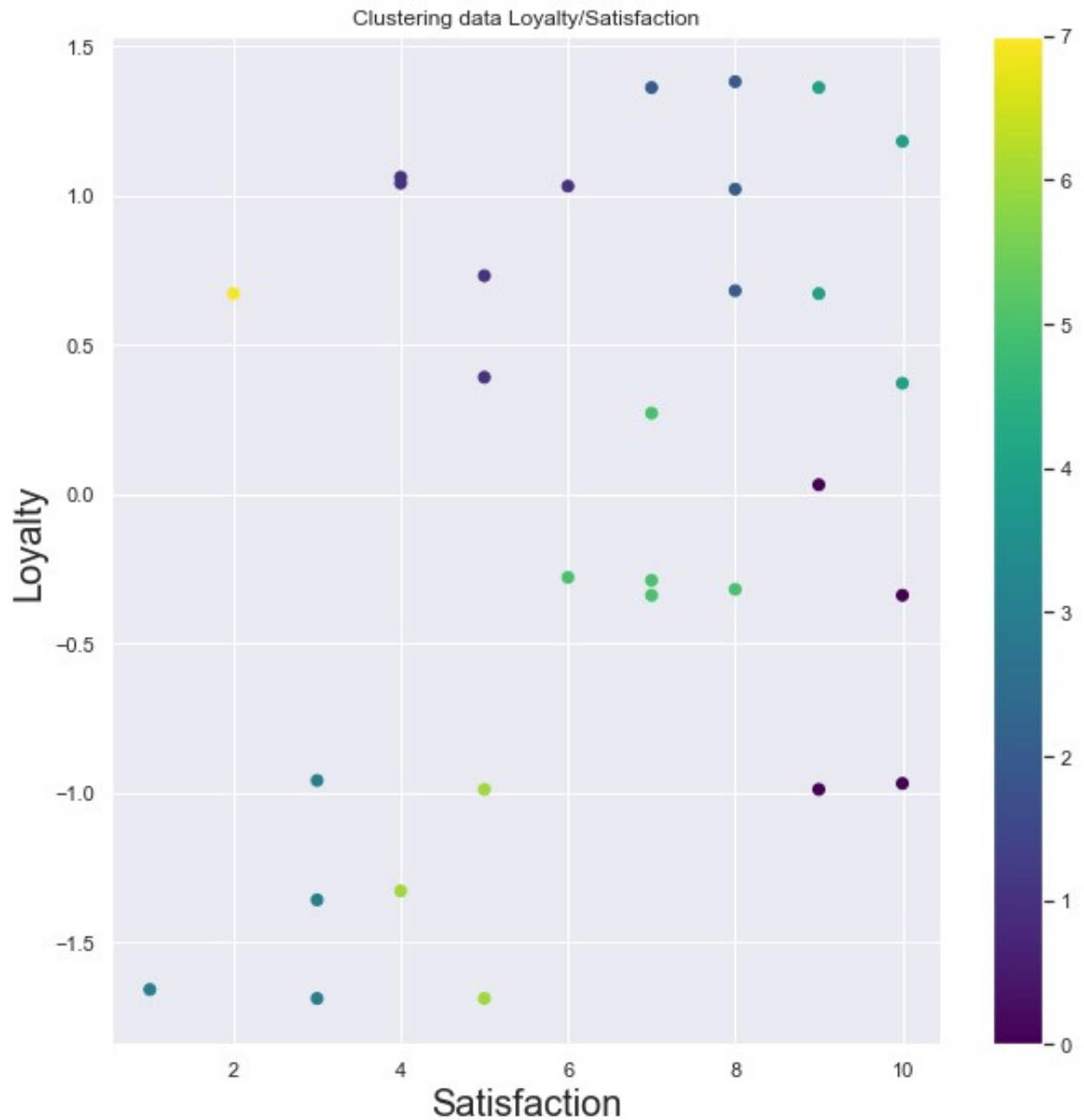
```
clusters =x.copy()
```

```
clusters['cluster_pred']= kmeans.fit_predict(x)
cmap = cm.get_cmap('viridis') # or another colormap like 'plasma',
'inferno', etc.
```

```
plt.figure(figsize=(10,10))
plt.scatter(clusters['Satisfaction'],
clusters['Loyalty'],c=clusters['cluster_pred'],cmap=cmap)
plt.xlabel('Satisfaction', fontsize=20)
plt.ylabel('Loyalty',fontsize=20)
plt.title('Clustering data Loyalty/Satisfaction')
plt.colorbar()
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_268\2686066597.py:6:
MatplotlibDeprecationWarning: Auto-removal of grids by pcolor() and
pcolormesh() is deprecated since 3.5 and will be removed two minor
releases later; please call grid(False) first.
    plt.colorbar()
```

```
<matplotlib.colorbar.Colorbar at 0x193a0667f70>
```



Standardize the variables

```
from sklearn import preprocessing
x_scaled=preprocessing.scale(x)
x_scaled
array([[ -0.93138063, -1.3318111 ],
       [-0.15523011, -0.28117124],
       [-0.54330537, -0.99160391],
       [ 0.23284516, -0.29117733],
       [-0.93138063,  1.05964534],
```

```

[-2.09560642, -1.6620122 ],
[ 1.39707095, -0.97159172],
[ 0.62092042, -0.32119561],
[ 0.62092042, 1.01962097],
[ 0.62092042, 0.67941378],
[ 1.39707095, -0.3412078 ],
[-0.54330537, 0.38923705],
[-0.54330537, -1.69203048],
[-1.70753116, 0.66940768],
[ 0.23284516, 0.26916393],
[ 1.00899568, 1.35982816],
[ 0.62092042, 1.37984035],
[ 0.23284516, 1.35982816],
[ 0.23284516, -0.3412078 ],
[ 1.00899568, 0.66940768],
[ 1.39707095, 1.17971847],
[-1.31945589, -1.69203048],
[-0.93138063, 1.03963316],
[-1.31945589, -0.96158562],
[-0.15523011, 1.02962706],
[ 1.00899568, -0.99160391],
[ 1.39707095, 0.36922486],
[ 1.00899568, 0.02901767],
[-1.31945589, -1.36182938],
[-0.54330537, 0.72944425]])

```

Take advantage of the Elbow method

```
wcss=[]
```

```

for i in range(1,10):
    kmeans=KMeans(i)
    kmeans.fit(x_scaled)
    inert= kmeans.inertia_
    wcss.append(inert)

```

```
wcss
```

```

C:\ProgramData\Anaconda3\lib\site-packages\sklearn\cluster\
_kmeans.py:1036: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(

```

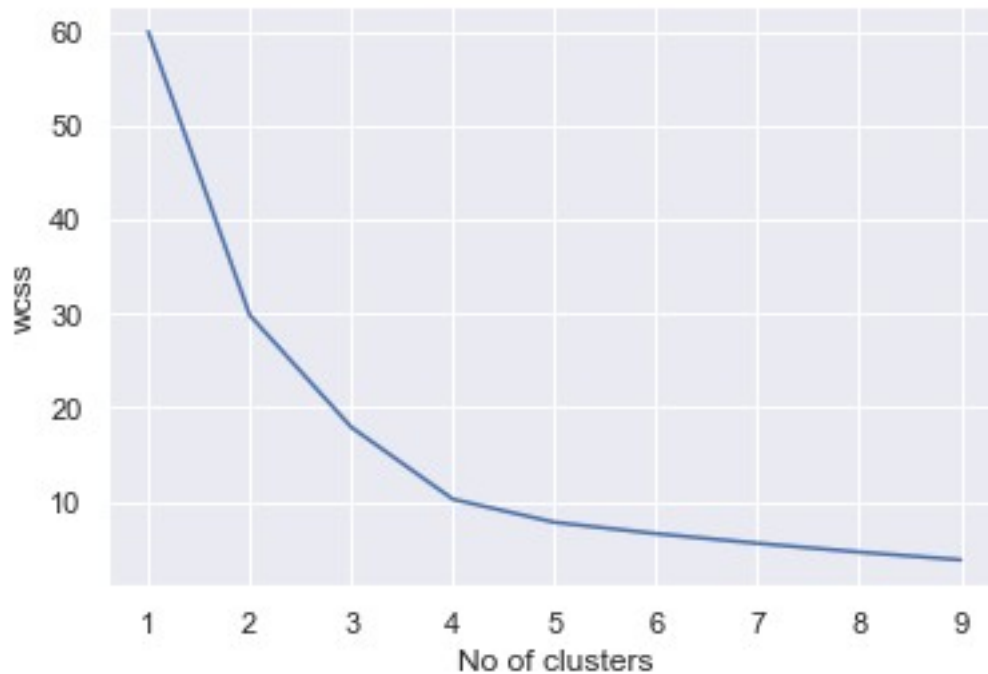
```

[59.99999999999999,
 29.818973034723143,
 17.913349527387965,
 10.24718180592842,
 7.792695153937187,
 6.586212092192188,

```

```
5.558520827037861,  
4.622832858801697,  
3.7978681376699055]
```

```
plt.plot(range(1,10),wcss)  
plt.xlabel('No of clusters')  
plt.ylabel('wcss')  
Text(0, 0.5, 'wcss')
```



From graph we can clearly see the 4 or 5 nos of cluster are best choice

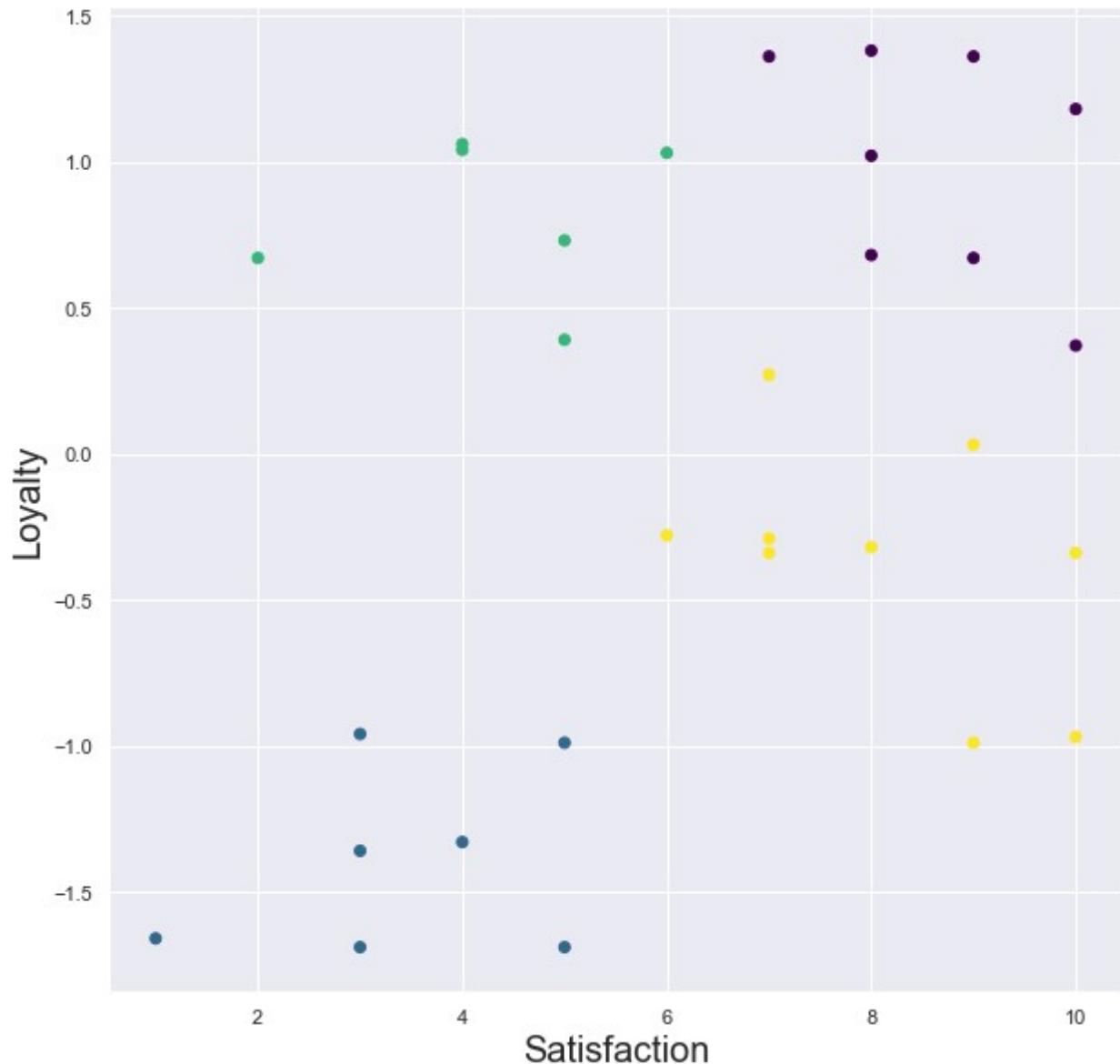
Plotting Cluster Chart with 4 clusters

```
kmeans_4=KMeans(4)  
kmeans_4.fit(x_scaled)  
clusters_new=x.copy()  
clusters_new['clusters_pred']=kmeans_4.fit_predict(x_scaled)  
clusters_new
```

	Satisfaction	Loyalty	clusters_pred
0	4	-1.33	1
1	6	-0.28	3
2	5	-0.99	1
3	7	-0.29	3
4	4	1.06	2
5	1	-1.66	1
6	10	-0.97	3

7	8	-0.32	3
8	8	1.02	0
9	8	0.68	0
10	10	-0.34	3
11	5	0.39	2
12	5	-1.69	1
13	2	0.67	2
14	7	0.27	3
15	9	1.36	0
16	8	1.38	0
17	7	1.36	0
18	7	-0.34	3
19	9	0.67	0
20	10	1.18	0
21	3	-1.69	1
22	4	1.04	2
23	3	-0.96	1
24	6	1.03	2
25	9	-0.99	3
26	10	0.37	0
27	9	0.03	3
28	3	-1.36	1
29	5	0.73	2

```
plt.figure(figsize=(10,10))
plt.scatter(clusters_new['Satisfaction'], clusters_new['Loyalty'],
c=clusters_new['clusters_pred'], cmap=cmap)
plt.xlabel('Satisfaction',fontsize=20)
plt.ylabel('Loyalty',fontsize=20)
Text(0, 0.5, 'Loyalty')
```



Plotting Graph with 5 clusters

```
kmeans_5=KMeans(5)
kmeans_5.fit(x_scaled)
clusters_new=x.copy()
clusters_new['clusters_pred']=kmeans_5.fit_predict(x_scaled)

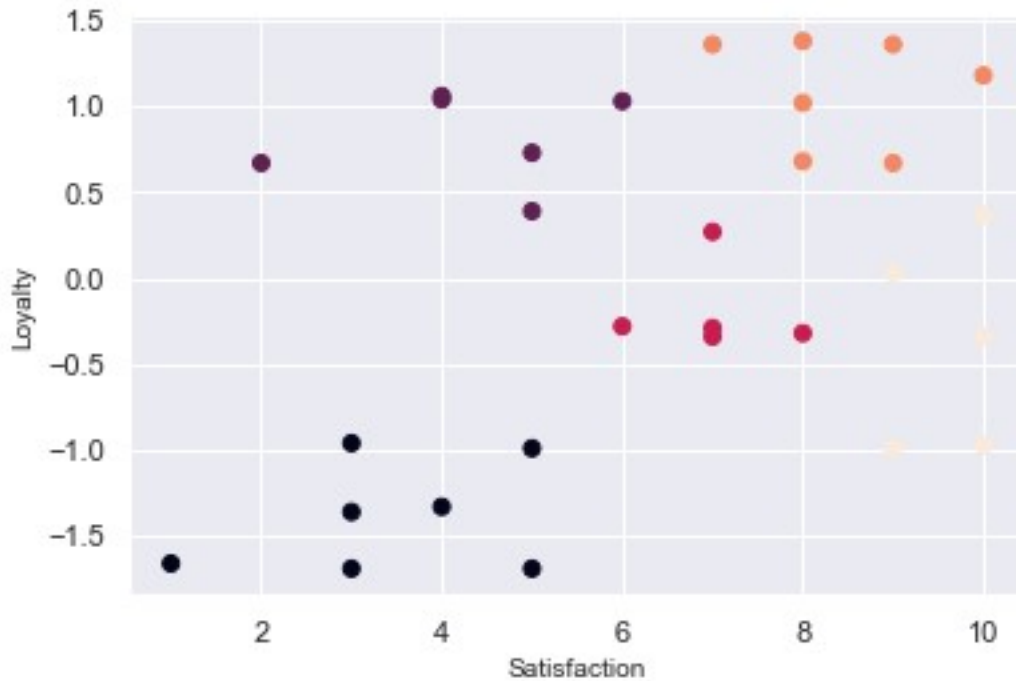
plt.figure(figsize=(10,10))

<Figure size 720x720 with 0 Axes>

<Figure size 720x720 with 0 Axes>

plt.scatter(clusters_new['Satisfaction'],clusters_new['Loyalty'],c=clusters_new['clusters_pred'])
```

```
plt.xlabel('Satisfaction', fontsize=10)
plt.ylabel('Loyalty', fontsize=10)
Text(0, 0.5, 'Loyalty')
```



From analysing both graphs we can choose one which may be more useful for our needs

In this case we can take 4 clusters and 4 type of customers

- 1.Fans- High satisfaction, high loyalty
- 2.Roamers- High satisfaction, low loyalty
- 3.Supporters- Low satisfaction, high loyalty
4. Alieanated- Low satisfaction, low loyalty